CPE 231 – Algorithms (1/2026)

Homework I Solution

1. (5 points) Design an algorithm that takes a list of n integers and produces an output list that cyclically right shift the input list by s positions, where $1 \leq s \leq n-1$. For example, suppose your input is $3, 4, 12, 9, 8, 7$ and $s = 3$, the output must be $9, 8, 7, 3, 4, 12$. Justify if it is a valid algorithm. You must not use an extra array other than that needed to store the input list.

Algorithm RightShift

Inputs: A sequence of numbers $a_1, a_2, \cdots, a_n$
        Amount of shifts $0 \leq s \leq n-1$
Output: The input sequence cyclically right shifted by s positions

for $i = 1$ to s **do**
    $aux \leftarrow a_n$
    for $j = n-1$ to 1 **do**
        $a_{j+1} \leftarrow a_j$
    $a_1 \leftarrow aux$

2. (15 points) Write down three different algorithms to determine the greatest common divisor (gcd) of two numbers m and n. Then, discuss their pros and cons.

Soln. The following three algorithms can be used to determine gcd(m,n) - Brute-force, Prime-factorization, Euclid's.

"Brute Force" Approach to gcd(m, n)

- Step 1:
  - If m or n is zero, return $\max\{m, n\}$.
  - Assign the value of $\min\{m, n\}$ to t.
- Step 2: Divide m by t.
  - If the remainder is 0, go to Step 3;
  - Otherwise, go to Step 4
- Step 3: Divide n by t.
  - If the remainder is 0, return t and stop;
  - Otherwise, go to Step 4
- Step 4: Decrease t by 1
  - If t is one, return t
  - Otherwise, go to Step 2

"Middle-School" Approach to gcd(m, n)

- Step 1: Find the prime factorization of m
- Step 2: Find the prime factorization of n
- Step 3: Find all the common prime factors
- Step 4: Compute the product of all the common prime factors and return it as gcd(m, n)

Euclid's Algorithm

- For $m \geq n$,
  - $\gcd(m, n) = \gcd(n, m \% n)$
  - Repeat until n becomes 0 and take m as the output
- Description
  - Step 1: If $n = 0$, return m and stop; otherwise go to Step 2
  - Step 2: Divide m by n and assign the value of the remainder to r
  - Step 3: Assign the value of n to m and the value of r to n. Go to Step 1.

Brute-force is easy to implement but less efficient as all the number less than equal to $\min(m, n)$ must be considered one by one. The prime factorization method is most complex to implement due to the factorization and finding the common factors. The euclid's algorithm is simple and most efficient one.

□

3. (10 points) From the algorithm below, work out the counts of basic operations $C(n)$ and its growth order.

Algorithm Mystery

```
1: Input: Positive integer n
2: Output:
3: r ← 0
4: for i := 1 to n − 1 do
5:     for j := i + 1 to n do
6:         for k := 1 to j do
7:             r ← r + 1
8: return r
```

Soln.

$$\begin{aligned}
\sum_{i=1}^{n-1}\sum_{j=i+1}^{n}\sum_{k=1}^{j}1 &= \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} j \\
&= \sum_{i=1}^{n-1}\left(\sum_{j=1}^{n} j - \sum_{j=1}^{i} j\right) \\
&= \sum_{i=1}^{n-1}\left(\frac{n(n+1)}{2} - \frac{i(i+1)}{2}\right) \\
&= \frac{1}{2}\sum_{i=1}^{n-1} n^2 + n - i^2 - i \\
&= \frac{1}{2}\left((n-1)n^2 + (n-1)n - \left(\frac{n(n+1)(2n+1)}{6} - n^2\right) - \left(\frac{n(n+1)}{2} - n\right)\right) \\
C(n) &= \frac{n(n(n+1))}{2} - \frac{n(n+1)(2n+1)}{12} - \frac{n(n+1)}{4}
\end{aligned}$$

Therefore, the growth order is n^3. □

4. (15 points) From the given algorithm,

Algorithm YouTryFigureItOut

```
1: Input: An array of positive integers A[1...n]
2: Output:
3:
4: for i = 1 to n − 1 do
5:     for j = 1 to n − i do
6:         if A[j + 1] < A[j] then
7:             swap A[j] and A[j + 1]
8: i ← 1
9: highest ← 0
10: while i ≤ n do
11:     l ← 1, v ← A[i]
12:     while i + l ≤ n and A[i + l] = v do
13:         l ← l + 1
14:     if l > highest then
15:         highest ← l, m_v ← v
16:     i ← i + l
17: return m_v
```

(a) (3 points) What task does it solve ? Explain from the algorithm.

Soln. The nested for loop is the bubble sort and the nested loop finds the most frequent element in the sorted list. So, the algorithm determines the mode of an unsorted input list. □

(b) (2 points) What are the basic operations in the algorithm ?

Soln. The basic operation in the for loop is the key comparison $A[j+1] < A[j]$ in line 6, and that of the while loop is the key comparison $A[i+l] = v$ in line 12. □

(c) (3 points) Are best-case and worst-case $C(n)$ (based on the key comparison) the same? When do they happen?

Soln. The number of times that the basic operations in line 6 and 12 execute do not depend on the input. The key comparison in line 6 is executed for every combination of i and j in the nested for loop. The key comparison in line 12 is executed for n times. □

(d) (7 points) Determine $C(n)$ in each scenario in (c) and draw conclusion on the algorithm efficiency. ☒Show the computation details.

Soln. For the nested for loop (Bubble sort),

$$C_1(n) = \sum_{i=1}^{n-1} \sum_{j=1}^{n-i} 1$$

$$
\begin{aligned}
&= \sum_{i=1}^{n-1} n - i \\
&= \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i \\
&= n(n-1) - \frac{n(n-1)}{2} = \frac{n(n-1)}{2} \in \Theta(n^2)
\end{aligned}
$$

□

For the nested while loop, $C_2(n) = n \in \Theta(n)$. Therefore,

$$C(n) = C_1(n) + C_2(n) \in \Theta(\max\{n^2, n\}) = \Theta(n^2)$$